

Random Date Generator - Test Report

Concise strategy, assumptions and Jira-style findings for a 1-hour exploratory testing exercise

System under test	https://codebeautify.org/generate-random-date
Scope	Only the random date generator. Other links and tools on the page were out of scope.
Timebox	1 hour, as requested in the assignment.
Environment	Windows 12; Chrome 148.0.7778.96; Edge 147.0.3912.98.
Support tooling	Manual exploratory testing supported by a lightweight Python/Playwright investigation script.

Test objective

Assess whether the generator produces valid, trustworthy and correctly formatted dates for user-defined input, with emphasis on validation, boundary behavior and output integrity.

Test strategy

Focus area	Reason
Input validation	Most user-visible risk comes from invalid count, dates, time and range values.
Boundary values	Date generators often fail around leap days, same-day ranges, reversed ranges and small time windows.
Output properties	Because output is random, exact values are less important than invariants: valid date, in range, correct format.
Format handling	Preset and custom formats are core features and likely parser failure points.
State handling	After invalid input, the UI should not leave misleading stale output visible.

Assumptions and open questions

Assumptions	Start and End Date are expected to be inclusive. Count should represent a positive integer. The browser local timezone is used for date input unless the selected format explicitly outputs UTC.
Open questions	What is the official maximum count? What date range is officially supported? Should ISO 8601 output be documented as UTC? Should custom formats support escaped literal text?
Out of scope	Other Code Beautify tools, navigation links, ads, account features, backend security testing and full accessibility audit.

Techniques used

Risk-based exploratory testing; boundary value analysis; equivalence partitioning; negative testing; lightweight automation for reproducibility; property-based thinking for random output validation.

Executive summary

Happy-path generation worked in several cases, including valid leap year 2024-02-29 and single-day ranges. The main risks are insufficient input validation, implicit JavaScript date normalization, fragile custom format parsing and stale output after failed validation.

ID	Severity	Finding
F-01	High	Missing robust validation for count
F-02	High	Invalid or edge-case dates are silently normalized
F-03	High	Reversed date range is accepted
F-04	Medium	Silent validation failures leave stale output visible
F-05	Medium	Custom format parser corrupts literal text

F-06	Medium	Two date format options produce identical output
------	--------	--

Jira-style findings

F-01 - Missing robust validation for count

Severity	High
Category	Input validation / performance / availability
Summary	The count field blocks letters but does not robustly validate numeric boundaries, integer-only values, or a safe maximum.
Preconditions	Random Date Generator page is open. Previous output may or may not exist.
Steps to reproduce	1. Set Count to 1000000. 2. Set Start Date to 2025-01-01 00:00:00. 3. Set End Date to 2025-12-31 23:59:59. 4. Click Generate. 5. Repeat with Count = 1.9, Count = 0 and Count = -1.
Expected result	Only positive integer values within a safe maximum should be accepted. Invalid values should be rejected before generation with a clear message.
Actual result	Count = 1000000 makes the browser tab unresponsive. Count = 1.9 generates 2 dates. Count = 0 clears the output without explanation. Count = -1 provides no visible feedback.
Impact	A user can make the browser tab unavailable. Numeric edge values are handled inconsistently, reducing trust in generated output.
Recommendation	Add min, max and integer-only validation. Reject invalid values before starting generation and use a safe maximum count.

F-02 - Invalid or edge-case dates are silently normalized

Severity	High
Category	Date handling / data correctness
Summary	The generator appears to rely on JavaScript Date normalization. Entered dates may silently produce different dates than intended.
Preconditions	Random Date Generator page is open.
Steps to reproduce	Case A: 1. Set Count to 5. 2. Set Start Date to 2025-02-29 00:00:00. 3. Set End Date to 2025-02-29 23:59:59. 4. Click Generate. Case B: 1. Set Start Date to 0050-01-01 00:00:00. 2. Set End Date to 0050-12-31 23:59:59. 3. Click Generate.
Expected result	Invalid calendar dates should be rejected. Edge-case years should either be generated exactly as entered or explicitly rejected as unsupported.
Actual result	2025-02-29 is silently generated as 2025-03-01. Year 0050 is generated as 1950 without warning.
Impact	Generated data may not match user input. This is a data correctness risk, especially for test data generation and boundary testing.
Recommendation	Validate parsed date components against original input. Avoid relying only on JavaScript Date auto-correction. Document supported year range.

F-03 - Reversed date range is accepted

Severity	High
Category	Input validation / business rule validation
Summary	The tool generates dates even when Start Date is later than End Date.
Preconditions	Random Date Generator page is open.
Steps to reproduce	1. Set Count to 5. 2. Set Start Date to 2025-12-31 00:00:00. 3. Set End Date to 2025-01-01 23:59:59. 4. Click Generate.
Expected result	Generation should be blocked and a message such as "Start date cannot be later than end date" should be shown.
Actual result	Dates are generated despite the logically invalid range. No clear validation feedback is shown.
Impact	Users may trust output that does not match intended constraints. The date range contract is broken.
Recommendation	Validate Start Date <= End Date before generation. Highlight affected fields and block output until the range is valid.

F-04 - Silent validation failures leave stale output visible

Severity	Medium
Category	Input validation / UI state / result integrity
Summary	When invalid input is entered, the tool may fail silently and leave previous generated output visible.
Preconditions	Generate any valid output first so that the output text area contains previous dates.
Steps to reproduce	Case A: 1. Set End Date to not-a-date. 2. Click Generate. Case B: 1. Set Start Date to 2025-06-15 14:30:99. 2. Set End Date to 2025-12-31 23:59:59. 3. Click Generate.
Expected result	The tool should reject invalid input, show a field-level error message, and clear or mark the existing output as stale.
Actual result	No visible validation message is shown. No new dates are generated. Previous output remains visible.
Impact	The user may incorrectly assume that the visible result belongs to the latest input. This creates a stale-data/result-integrity risk.
Recommendation	On validation failure, show a precise message, keep focus near the invalid field, and clear or visually mark previous output as outdated.

F-05 - Custom format parser corrupts literal text

Severity	Medium
Category	Custom formatter / parsing / usability
Summary	The custom format parser appears to use simple token replacement. Literal words containing d, h, m or s can be modified unexpectedly.
Preconditions	Use a fixed date: Start Date = End Date = 2025-06-15 14:30:45.
Steps to reproduce	1. Set Count to 1. 2. Select Custom format. 3. Enter custom format Monday and click Generate. 4. Repeat with second and ssss.
Expected result	Literal text should be preserved, or the UI should document supported tokens and escaping rules clearly.
Actual result	Monday becomes Mon15ay. second becomes 45econ15. ssss becomes 4545s.
Impact	Users may receive corrupted output when combining literal labels with date/time tokens. Custom format behavior is not predictable.
Recommendation	Implement a deterministic tokenizer using explicit tokens such as YYYY, MM, DD, HH, mm, ss. Provide escaping rules for literal text.

F-06 - Two date format options produce identical output

Severity	Medium
Category	Functional correctness / format option mapping
Summary	Two different dropdown options produce the same output, suggesting a duplicate mapping or incorrectly implemented option.
Preconditions	Use a fixed date where day and month differ: Start Date = End Date = 2025-06-07 14:03:02.
Steps to reproduce	1. Set Count to 1. 2. Select Year Month Date HH MM SS and click Generate. 3. Note the output. 4. Select Year Date Month HH MM SS and click Generate. 5. Compare both outputs.
Expected result	Year Month Date should output 2025 June 07 14:03:02. Year Date Month should output 2025 07 June 14:03:02 or equivalent day/month ordering.
Actual result	Both options output 2025 June 07 14:03:02.
Impact	One format option appears redundant or wrong. Users relying on a specific output format may receive a different order than selected.
Recommendation	Fix the Year Date Month mapping or remove the duplicate option. Add regression checks for each dropdown format.

Additional observation - ISO 8601 timezone behavior

Observation	ISO 8601 output appears to be rendered in UTC, while input fields are entered without explicit timezone information.
Example	For input 1800-01-01 00:00:00 to 1800-01-01 00:00:03, output appeared as 1799-12-31T22:36:00Z to 1799-12-31T22:36:03Z. For 2025-01-01, output appeared one hour behind local input, e.g. 2024-12-31T23:00:00Z.
Assessment	Not reported as a bug. This is plausible if local input is converted to UTC for ISO output. The timezone assumption is simply not visible to the user.
Open question	Should the UI state that ISO 8601 output is UTC, or should the user be able to choose local vs UTC output?

Root cause themes

1. Insufficient input validation before generation.
2. Over-reliance on JavaScript Date normalization.
3. Fragile custom format token parsing.
4. Unclear UI state after validation failures.
5. Missing regression coverage for predefined output formats.

Recommended fix priority

1. Add strict input validation for count and date ranges.
2. Reject invalid calendar dates before generation.
3. Clear or mark output as stale after failed validation.
4. Replace custom format string replacement with a proper tokenizer.
5. Add automated checks for each dropdown format and key boundary date.